Machine Guardian

A motor-fault model for the MAC Attack INT8 accelerator

The whole system from start to finish: the data, the features, training, the experiments, how well the model works, the INT8 conversion and the check on the chip's RTL.

27 / 28 faulty test recordings found	**0 / 4** false alarms on healthy test recordings	**0.982** balanced accuracy (test)
1.000 ROC-AUC (test)	**2.64 us** on the chip per 1-s window	**0** 10-s decisions changed by INT8

Model E3, seed 42, on the test split (speed codes not used in training). Over the three training seeds: 25 to 27 of 28 found, no false alarm, balanced accuracy 0.970 ± 0.017. These are development results: the test split also guided which experiment came next (section 7).

Status, 27 October 2026: the model and its INT8 export are complete and checked bit for bit on the RTL in simulation; the choice of E3 is **proposed, for Alex to confirm**; not yet run on the board. Branch `alex/machine-guardian-model`.

MAC Attack capstone team. Prepared with Claude (Anthropic) from the team's spec. The model results, tables and charts are read from the saved result files when the report is built (`model/report/make_report.py`); the RTL, board and independent-check facts are quoted from `docs/log.md` section 12.

1. Summary

What it does. Machine Guardian listens to an electric motor through vibration sensors and says whether the motor is healthy or faulty. Once a second it takes the last second of vibration from three accelerometers, reduces it to 22 numbers, and runs a small neural network on the team's INT8 systolic-array accelerator (the TPU). The ARM processor adds up the last 10 answers, so the decision always covers the last 10 seconds (output 1). A second output, the health index, says how far the vibration is from the healthy motor's at the same speed, from 0 to 100% (output 2).

How well it works. On the test recordings (speed codes not used in training), the proposed model (E3, seed 42) found **27 of the 28 faulty recordings** and raised **no false alarm** on the 4 healthy ones: accuracy 0.969, balanced accuracy 0.982, precision 1.000, recall 0.964, F1 0.982, MCC 0.878, ROC-AUC 1.000. All three training seeds raise no false alarm and find 25 to 27 of 28 (balanced accuracy 0.970 ± 0.017, MCC 0.824 ± 0.077). James's baseline, rebuilt as E0, found 18 of 28. The INT8 version changes no 10-second decision, and the accelerator's RTL produces every logit bit for bit.

Output 2 works where the healthy reference has data: on the 30->60 Hz test ramp both healthy recordings score 100% and the faulty median is 0%, and the validation ROC-AUC is 0.955. At 15 Hz it fails (both healthy test recordings score 0%; test ROC-AUC 0.527).

How far to trust it.

- The dataset has only one healthy motor, and on accelerometers 1 and 2 it is quieter than every faulty recording at every speed. The model may partly be recognizing that motor rather than the fault, and accelerometer 1, the sensor E3 adds, carries the same signal.
- The test split was not only used at the end: E3 was run because E2 missed test faults, and the rule that picked E3 over E2 was set after both test results were known. The three-seed mean is the less optimistic figure.
- The test set has only 4 healthy recordings, so a false-alarm rate as high as 60% cannot be ruled out. Every mistake is at 15 Hz, the one test speed below every training window.
- In INT8 the healthy 15-Hz recordings sit much closer to the threshold than in FP32 (-6.3 and -9.9 instead of -27.1 and -20.1; the cut is 1.11).

Where each part is. Section 2 walks through the system. Sections 3-6 cover the data, the features, the drive frequency and training. Section 7 covers the four experiments and the choice of E3. Section 8 has every quality measure. Sections 9-11 cover the health index, the INT8 conversion and the chip. Section 12 lists the independent checks, section 13 the limits.

2. The system from start to finish

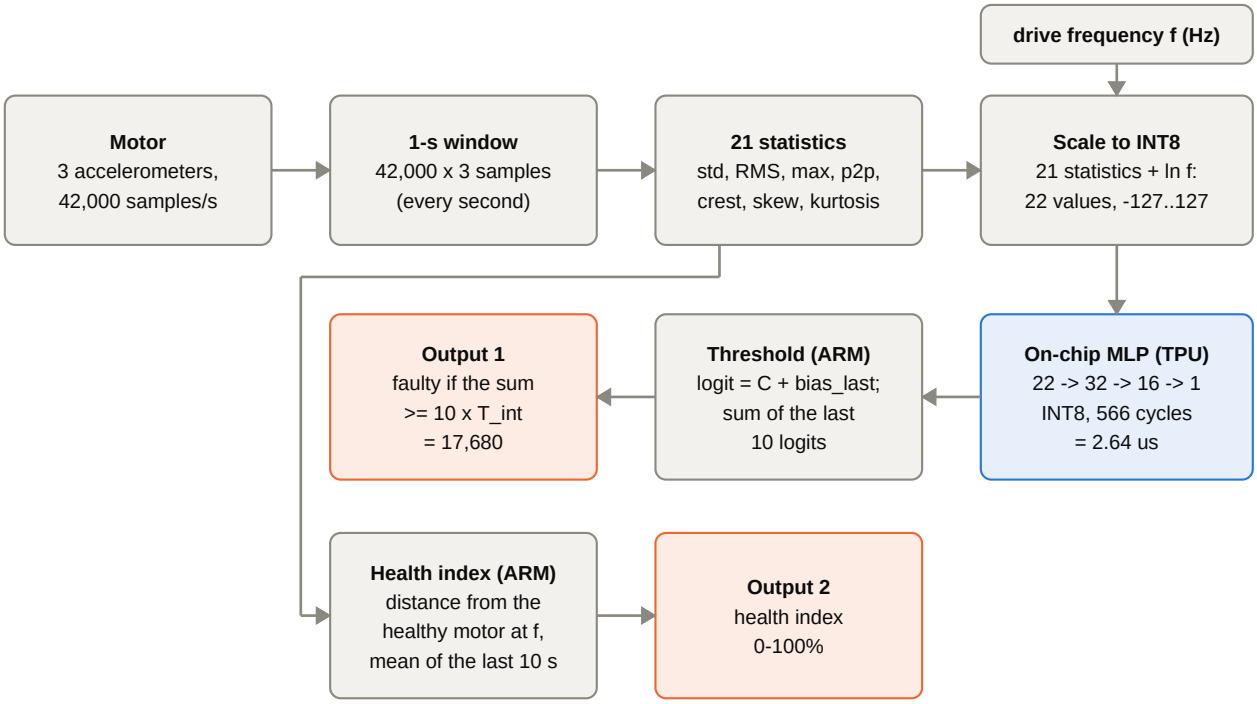


Figure 1. Machine Guardian, once a second. The blue box runs on the accelerator; the grey boxes after the sensors run on the ARM processor; the orange boxes are the two outputs.

- **Sense.** Three accelerometers on the motor, sampled at 42,000 samples per second, and the frequency f the motor drive is set to (from the drive, not measured).
- **Window.** Every second, the last 1 s of samples (42,000 per sensor).
- **Describe.** 7 statistics per accelerometer: the logarithm of the standard deviation, RMS, largest absolute value and peak-to-peak, then crest factor, skewness and kurtosis. With ln(f) that is 22 inputs.
- **Scale.** Each input is standardized with the training data's mean and spread, divided by one input scale s_x = 0.0475667 and rounded to an integer from -127 to 127: one row of INT8 values.
- **Run on the chip.** The accelerator runs the three layers (22 -> 32 -> 16 -> 1, ReLU between them, 1,232 INT8 weights) in 566 clock cycles, 2.64 us at 214.286 MHz, and returns the last layer's raw integer sum C.
- **Decide (output 1).** The ARM adds the last bias (-120) to get an integer logit. One integer step is worth 0.000626296 of a real logit, so the trained threshold, cut = 1.1071, becomes T_int = 1,768 integer steps. The ARM keeps the last 10 logits and calls the motor faulty when their sum reaches 10 x T_int = 17,680.
- **Health index (output 2).** From the same statistics and f, the ARM measures how far each window is from the healthy motor at that speed and averages the last 10 seconds into a 0-100% index (section 9; not reliable at low speed).

3. The data

The model is trained on the **University of Ottawa electric motor dataset** (vibration). It has 128 recordings of 10 s each at 42 kHz, with three accelerometers, a microphone and a temperature sensor. Only the accelerometers are used. There are 8 classes of 16 recordings each: one healthy motor (`H_H`) and 7 fault classes, named by the dataset's codes (`B_R, F_B, K_A, R_M, R_U, S_W, V_U`); the model only learns healthy against faulty. Every class is recorded at 8 speed settings and 2 loads. File names are class_speedcode_load: `H_H_5_0` is the healthy motor at speed code 5, unloaded. Speed codes 1-4 are constant (15, 30, 45, 60 Hz); codes 5-8 ramp over the 10 s (15->45, 30->60, 45->15, 60->30 Hz). Label 0 = healthy, 1 = faulty; the positive class is faulty.

The split is fixed by speed code, as in James's earlier work on this dataset (his baseline model, rebuilt here as E0). No test speed code is used in training. For the window models E2 and E3, which take each window's drive frequency as an input, only 15 Hz lies outside the training range of 16.5-60 Hz: the 30->60 Hz test ramp covers the same frequencies as the 60->30 Hz training ramp. All windows of a recording stay in its recording's split; the standardizer and the class weight come from the training split only.

Split	Speed codes (drive frequency)	Healthy	Faulty	1-s windows
train	3 (45 Hz), 4 (60 Hz), 5 (15->45 Hz), 8 (60->30 Hz)	8	56	640
val	2 (30 Hz), 7 (45->15 Hz)	4	28	320
test	1 (15 Hz), 6 (30->60 Hz)	4	28	320

A warning built into the data: the healthy motor is quieter. On accelerometers 1 and 2 the one healthy motor is quieter than every faulty recording at every speed (ratio above 1x); on accelerometer 3 the ranges overlap at speed codes 4 and 8. A model can therefore separate the classes by recognizing this motor, not the fault. Ratio = quietest faulty recording / loudest healthy one (healthy range vs faulty range, m/s^2, both loads). Accelerometer 1 carries a constant offset of about 5 m/s^2, so its spread (std) is the comparable measure.

Speed code	Accel. 1 (std)	Accel. 2 (RMS)	Accel. 3 (RMS)
1 (15 Hz)	**1.49x** (1.62-1.64 vs 2.43-5.46)	**1.32x** (0.56-0.60 vs 0.79-2.36)	**1.17x** (0.58-0.63 vs 0.73-2.38)
2 (30 Hz)	**2.12x** (2.26-3.22 vs 6.83-16.46)	**1.43x** (1.70-1.99 vs 2.84-4.84)	**1.66x** (1.55-1.76 vs 2.91-5.42)
3 (45 Hz)	**2.52x** (3.05-6.42 vs 16.18-32.35)	**2.06x** (3.48-3.79 vs 7.82-12.15)	**1.33x** (3.31-3.46 vs 4.59-8.28)
4 (60 Hz)	**2.14x** (5.06-8.64 vs 18.53-61.37)	**2.47x** (3.17-3.99 vs 9.88-13.98)	**0.91x** (4.76-11.56 vs 10.51-16.23)
5 (15->45 Hz)	**2.77x** (4.06-4.49 vs 12.45-20.18)	**2.65x** (1.77-1.97 vs 5.20-7.41)	**1.75x** (1.77-1.89 vs 3.32-5.32)
6 (30->60 Hz)	**3.25x** (4.12-7.16 vs 23.30-43.60)	**2.93x** (2.90-3.04 vs 8.91-14.59)	**1.45x** (3.67-5.04 vs 7.29-11.86)
7 (45->15 Hz)	**2.26x** (4.09-4.20 vs 9.49-17.31)	**2.15x** (1.54-2.00 vs 4.30-6.15)	**1.75x** (1.67-1.68 vs 2.94-4.63)
8 (60->30 Hz)	**2.02x** (3.57-7.15 vs 14.41-44.72)	**1.91x** (2.75-3.17 vs 6.06-14.66)	**0.58x** (2.71-6.67 vs 3.89-12.09)

4. Features

Each CSV file is read once. For every 10-s recording and every 1-s window (42,000 samples), `features.py` computes 8 statistics per accelerometer in float64: mean, standard deviation, RMS, largest absolute value (max|x|), peak-to-peak, crest = max|x| / RMS, skewness m3/std^3 and kurtosis m4/std^4. They are checked against SciPy (within 6e-16) and the windows add back up to their recording (within 7e-12).

Experiment	Rows	Inputs	Sensors
E0 (James's baseline)	10-s recordings	16: the 8 statistics as they are	accel. 2, 3
E1	10-s recordings	15: ln of std, RMS, max|x|, p2p; crest, skew, kurtosis; ln(f), f = the nominal speed	accel. 2, 3
E2	1-s windows	15: as E1, with f = the window's drive frequency	accel. 2, 3
E3 (proposed)	1-s windows	22: as E2, for all three accelerometers	accel. 1, 2, 3

Why logarithms: vibration amplitude grows roughly as a power of speed, which becomes a straight line in ln(f), something a ReLU network can continue past the training speeds. Why windows: 10 times as many examples, each with one speed, and the board can decide once a second.

5. The drive frequency f and the ramp check

E2 and E3 need the drive frequency of every 1-s window. For the four ramp codes the spec assumed a straight ramp over the 10 s, so it asked for a check first: a spectrogram of `H_H_5_0.csv`. `speed.py` follows the shaft-rotation line (1x) of every ramp recording and fits a ramp-and-hold command to it: the code's start frequency, a straight ramp, then the end frequency. Only the start and end times are fitted.

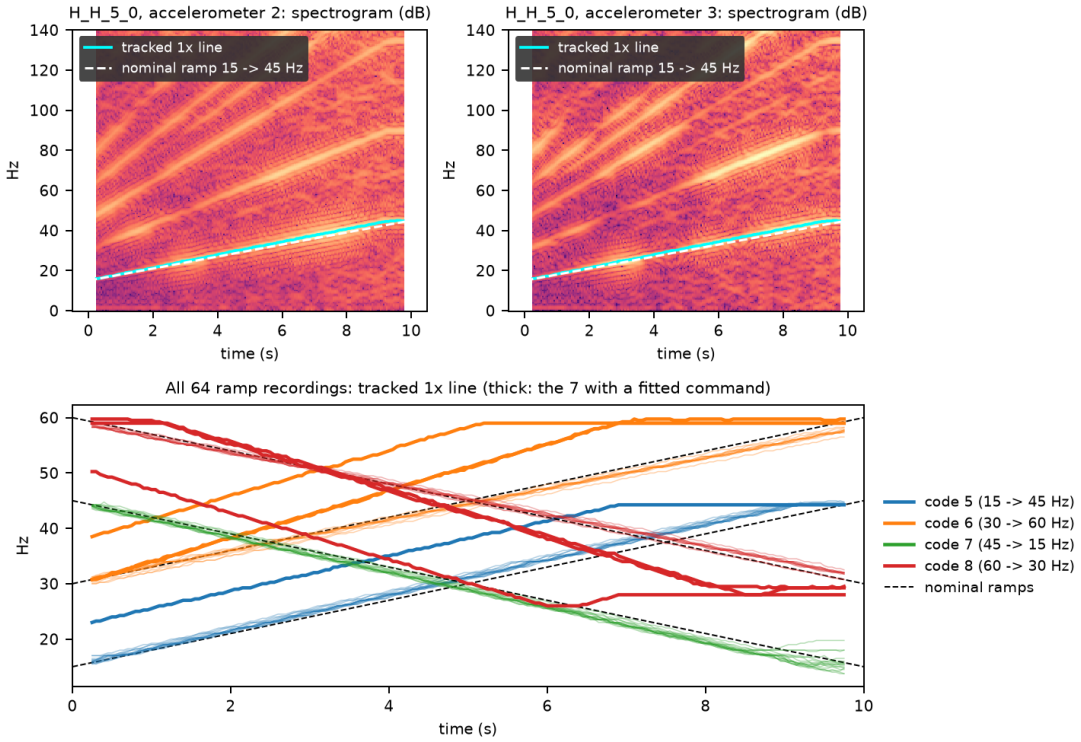


Figure 2. Top: H_H_5_0 (accelerometers 2 and 3) with the tracked 1x line and the nominal ramp - it is linear. Bottom: the 1x line of all 64 ramp recordings, coloured by speed code; thick lines are the 7 that did not follow the nominal ramp (named in the table below).

Result. 57 of the 64 ramp recordings have a fitted command within 2.85 Hz of the nominal ramp at every moment of the 10 s (one of them, R_M_7_0, turns back up in its last second; its last window keeps the nominal value, about 3 Hz low; accepted). 7 do not: the drive ramped faster and held, or the recording started in the middle of the ramp. Those 7 use their fitted command; every other recording keeps the nominal value. A negative start time means the ramp began before the recording.

Recording	Split	Fitted ramp	Largest distance from nominal	10-s mean f (nominal)
S_W_5_0	train	-2.30 s -> 7.15 s	8.55 Hz	36.9 Hz (30)
H_H_6_0	test	+0.10 s -> 7.10 s	8.70 Hz	49.2 Hz (45)
S_W_6_0	test	-1.80 s -> 5.40 s	13.80 Hz	53.9 Hz (45)
V_U_6_0	test	+0.05 s -> 7.10 s	8.70 Hz	49.3 Hz (45)
H_H_8_0	train	+0.90 s -> 7.90 s	6.30 Hz	43.2 Hz (45)
S_W_8_0	train	-2.10 s -> 5.05 s	14.85 Hz	35.4 Hz (45)
V_U_8_0	train	+1.00 s -> 8.05 s	5.85 Hz	43.6 Hz (45)

Why not just measure the shaft speed? The measured speed includes the motor's slip, and the healthy motor turns slightly faster than most faulty recordings. At 60 Hz it is faster than every one, so the measured speed alone would identify the healthy motor. f therefore always uses the commanded levels; on the board it comes from the drive, never from a shaft-speed sensor.

Speed code	Healthy motor (2)	Faulty (14)	Faulty at or above the slowest healthy
1 (15 Hz)	15.31-15.42 Hz	14.63-15.32 Hz	1
2 (30 Hz)	30.66-30.71 Hz	29.38-30.74 Hz	1
3 (45 Hz)	44.69-44.75 Hz	44.32-45.35 Hz	3
4 (60 Hz)	59.79 Hz	59.08-59.23 Hz	0

6. Training

Setting	Value
Network	MLP, inputs -> 32 -> 16 -> 1, ReLU, no dropout (the chip allows at most 64 per layer)
Loss	BCEWithLogitsLoss, pos_weight = train healthy / train faulty = 8/56 (80/560 windows)
Optimizer	Adam, learning rate 1e-3, betas (0.9, 0.999), eps 1e-8, no weight decay
Batches	64 rows; batch order from torch.randperm with a generator seeded like the model
Stopping	at most 100 epochs; stop after 15 epochs without a validation-loss improvement > 1e-5; keep the best epoch's weights
Seeds	42, 43, 44 (42 is the reported and exported run); deterministic on the CPU
Recording score	the logit (E0, E1) or the mean of its 10 window logits (E2, E3)
Threshold	the lowest cut among the validation scores that maximizes validation F1 with validation recall >= 0.8; faulty if score >= cut

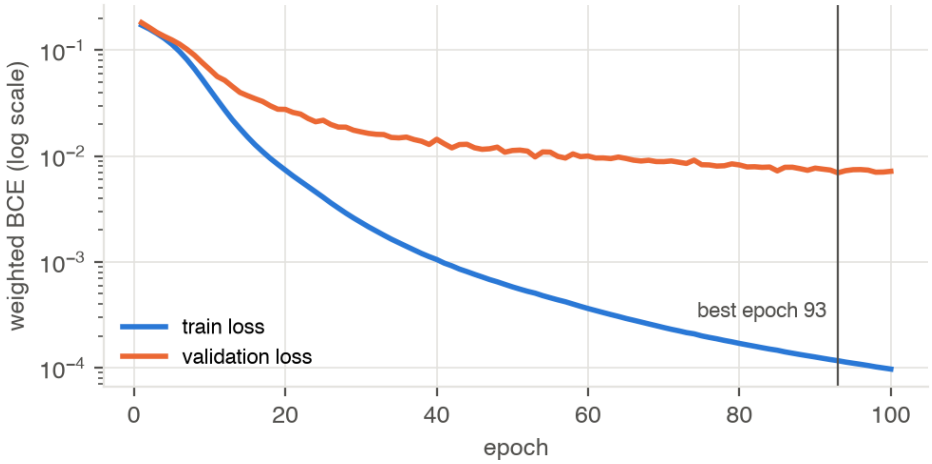


Figure 3. E3 seed 42: training and validation loss per epoch. The best validation loss is at epoch 93 of 100; those weights are kept. Its threshold is cut = 1.1071 (as a probability, t = sigmoid(cut) = 0.7516).

10 of the 12 runs (E0-E3, three seeds each) reached the 100-epoch limit, most with their best epoch near the end, so the validation loss was still falling slowly; the spec sets the limit and longer runs were not tried. Every run is reproducible: the same command gives byte-identical weights and results. An independent trainer written from the spec reproduced every weight of E1-E3 once four details the spec left unstated were supplied (now in `train.py`'s docstring).

7. The experiments and the choice of model

Four experiments, each defined before it was run and never changed after its test result was seen. E0 rebuilds James's baseline; E1 adds speed; E2 moves to 1-s windows; E3 adds accelerometer 1. E3 was run because E2 seed 42 still missed 4 test faults (none on validation), as the spec asks.

Experiment	Seed	Test faults found	False alarms	Test ROC-AUC	Val. loss (per recording)	Best / last epoch
E0	42	18 / 28	0 / 4	0.875	0.0864	100 / 100
	43	14 / 28	0 / 4	0.768	0.1552	32 / 47
	44	16 / 28	0 / 4	0.893	0.0984	100 / 100
E1	42	15 / 28	0 / 4	0.929	0.0539	100 / 100
	43	17 / 28	2 / 4	0.562	0.0499	100 / 100
	44	15 / 28	0 / 4	0.973	0.0677	100 / 100
E2	42	24 / 28	0 / 4	1.000	0.0004	99 / 100
	43	17 / 28	2 / 4	0.580	0.0657	11 / 26
	44	23 / 28	0 / 4	1.000	0.0004	98 / 100
E3	42	27 / 28	0 / 4	1.000	0.0015	93 / 100
	43	25 / 28	0 / 4	1.000	0.0017	97 / 100
	44	27 / 28	0 / 4	1.000	0.0018	98 / 100

Validation loss on one common scale: the weighted BCE of the 32 validation recording scores. (E2 and E3 stop training on the per-window loss, which is not comparable with E0 and E1's.)

E0 reproduces James's result exactly (18/28, 0/4, ROC-AUC 0.875). **E1** does not fix 15 Hz (15/28). **E2** finds 24/28, but its seed 43 stopped early and is poor. **E3**: every seed ranks every test recording correctly and raises no false alarm, finding 27, 25, 27 of 28.

Why E3, stated plainly. On validation all of E1, E2 and E3 are perfect (F1 and ROC-AUC 1 in every seed), so only the validation loss is left. On the common scale above, E1 is the worst in every view (mean 0.057). Between E2 and E3 the rule written before E1 (seed 42 and the mean of the three seeds) disagrees with itself: seed 42 alone prefers E2 (0.0004 against 0.0015), the mean prefers E3 (0.0017 against 0.0222, because E2's seed 43 is poor). Ranking the mean and the worst seed first was decided after the E2 and E3 test results had been seen, so the test result may have influenced it. The reason to prefer E3 that does not use test is its stability: every seed gives nearly the same model. E2 seed 42 (15 inputs, two sensors) is the alternative under the seed-42 rule. The choice is **proposed, for Alex to confirm**.

8. How good is the model?

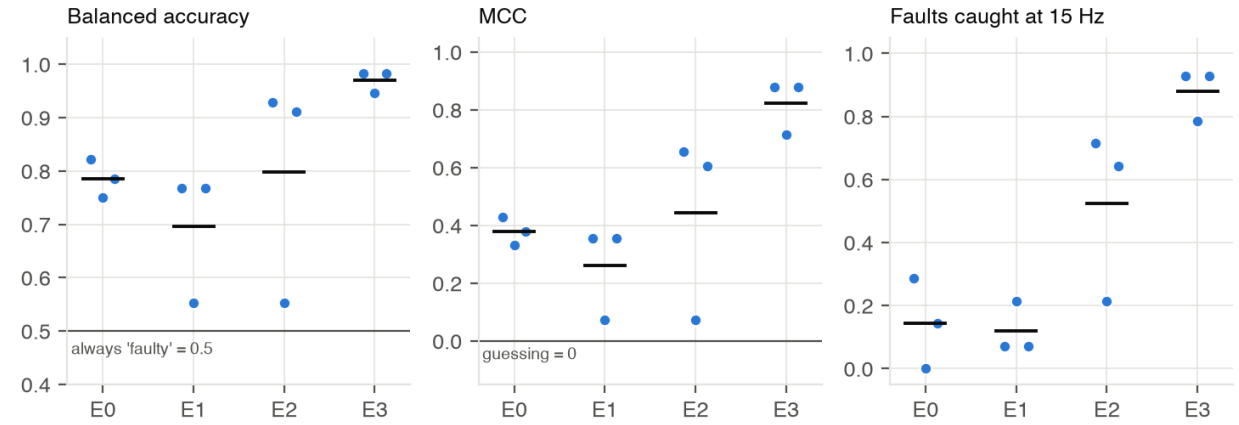
Reading the measures. *Recall* = share of faulty recordings caught. *Specificity* = share of healthy recordings cleared. *Precision* = share of 'faulty' calls that were right. *Balanced accuracy* = the average of recall and specificity. *MCC* (Matthews correlation) runs from -1 to 1, 0 for guessing. *ROC-AUC* = the chance a random faulty recording scores higher than a random healthy one (ranking, independent of the threshold). The test set is 28 faulty and 4 healthy recordings, so a model that calls everything faulty already scores 0.875 accuracy and 0.933 F1. **Balanced accuracy and MCC are the fair measures here.**

Seed 42, test	Accuracy	Balanced	Precision	Recall	Specificity	F1	MCC	ROC-AUC
Always 'faulty' (no model)	0.875	0.500	0.875	1.000	0.000	0.933	0	-
E0	0.688	0.821	1.000	0.643	1.000	0.783	0.429	0.875
E1	0.594	0.768	1.000	0.536	1.000	0.698	0.355	0.929
E2	0.875	0.929	1.000	0.857	1.000	0.923	0.655	1.000
E3	0.969	0.982	1.000	0.964	1.000	0.982	0.878	1.000

E0 and E1 have lower accuracy than calling everything faulty. E3 is clearly above that rule on every measure except recall, which the rule gets by definition.

Model	Balanced acc.	Recall	Specificity	MCC	ROC-AUC	Caught at 15 Hz (42 / 43 / 44)
E0	0.786 ± 0.029	0.571 ± 0.058	1.000 ± 0.000	0.380 ± 0.039	0.845 ± 0.055	0.29 / 0.00 / 0.14
E1	0.696 ± 0.101	0.560 ± 0.034	0.833 ± 0.236	0.261 ± 0.133	0.821 ± 0.184	0.07 / 0.21 / 0.07
E2	0.798 ± 0.173	0.762 ± 0.110	0.833 ± 0.236	0.444 ± 0.264	0.860 ± 0.198	0.71 / 0.21 / 0.64
E3	0.970 ± 0.017	0.940 ± 0.034	1.000 ± 0.000	0.824 ± 0.077	1.000 ± 0.000	0.93 / 0.79 / 0.93

Test, mean ± population standard deviation over the seeds 42, 43, 44. By balanced accuracy and MCC, seed 42 is the best or tied-best seed in every experiment, so its single-seed numbers are the optimistic end.



Test split. Each dot is one seed (left to right: 42, 43, 44); black line = mean of the three.

Figure 4. Every seed of every model on test. E3's three seeds sit together near the top; E2 depends on the seed.

Where the mistakes are

Every model catches all 14 faulty recordings at speed code 6 (30->60 Hz), in every seed. Every mistake is at speed code 1 (15 Hz), below every training window's speed (16.5 Hz and up; E1's 10-s speeds start at 30 Hz, and E0 has no speed input). The test therefore mostly measures how well each model handles an unseen low speed. E3 seed 42 misses one recording, R_U_1_1.



Figure 5. E3 seed 42 in FP32: the score of every validation and test recording (mean logit of its 10 windows) against the threshold. Above the line = called faulty. The healthy and faulty recordings are fully separated (ROC-AUC 1.000); the one miss scores -1.93, 3.0 below the cut but far above every healthy recording (highest -8.1). In INT8 the two healthy 15-Hz recordings move up to -6.3 and -9.9 (section 10).

How sure can we be?

With so few test recordings, the measured rates come with wide uncertainty (95% Clopper-Pearson intervals). E3's fault detection is reasonably pinned down; the false-alarm rate is not: 0 false alarms in 4 healthy recordings is consistent with a true rate anywhere from 0 to 60%.

Seed 42	Recall (faults caught)	95% interval	False-alarm rate	95% interval
E0	18 / 28 = 0.643	0.44 - 0.81	0 / 4	0.00 - 0.60
E1	15 / 28 = 0.536	0.34 - 0.72	0 / 4	0.00 - 0.60
E2	24 / 28 = 0.857	0.67 - 0.96	0 / 4	0.00 - 0.60
E3	27 / 28 = 0.964	0.82 - 1.00	0 / 4	0.00 - 0.60

Validation (all 1.000 for E1-E3) is not independent evidence: the threshold is set at the lowest faulty validation score, so validation recall is 1 by construction, and early stopping used the validation loss. And the test split itself steered the experiments (section 7), so these test numbers are development results, not a final hold-out.

How quickly it can decide

The board decides from the last 10 s, but the model is already useful sooner. For every block of k consecutive 1-s windows of every test recording, the mean logit was compared with the same threshold (FP32). For E3, a single second already catches about 95% of the faulty blocks with no false alarm; averaging 10 s adds little. The blocks overlap and come from the same 32 recordings, so they are not independent samples. In INT8, E3 seed 42 gives the same recall except at 2 s (0.956 instead of 0.948), still with no false alarm.

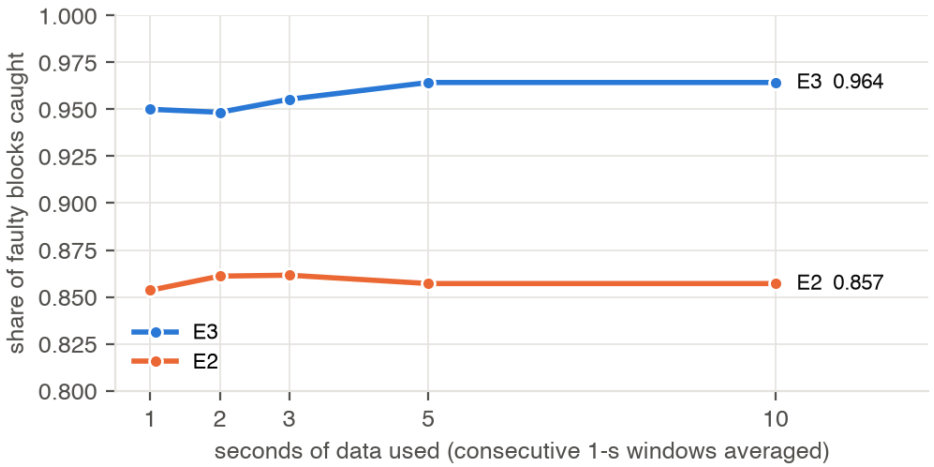


Figure 6. Share of faulty test blocks caught against seconds of data, seed 42 (FP32). Neither model raised a false alarm at any block length.

Seconds of data	E3 s42 recall	E3 s43	E3 s44	E2 s42 recall	E2 s43	E2 s44	Faulty / healthy blocks
1	0.950	0.907	0.961	0.854	0.611	0.850	280 / 40
2	0.948	0.921	0.964	0.861	0.587	0.853	252 / 36
3	0.955	0.920	0.969	0.862	0.594	0.862	224 / 32
5	0.964	0.917	0.964	0.857	0.589	0.851	168 / 24
10	0.964	0.893	0.964	0.857	0.607	0.821	28 / 4

Recall = share of the faulty blocks caught. Lowest specificity over the block lengths: E3 s42 1.00, E3 s43 1.00, E3 s44 1.00, E2 s42 1.00, E2 s43 0.50, E2 s44 1.00.

9. Output 2: the health index

The professor asked for a second output: the motor's quality from 0 to 100%. The dataset has no wear levels (one healthy motor, each fault at a single severity), so '50% = half its capability' can be neither trained nor tested. The team chose a **health index** instead: 100% = inside the healthy motor's range at that speed, 0% = as far out as a typical faulty recording. It is a distance from the one healthy motor, not a remaining capability.

How it is computed (no training, no seeds). For each of the 21 window statistics, a straight line in ln(f) is fitted to the 80 healthy training windows, with the healthy spread around it. A window's distance d is the root-mean-square of its 21 standardized deviations; D is the mean of the last 10 windows. The anchors come from training only: D_100 = 2.508 (the largest healthy distance, each measured at a speed left out of the fit) and D_0 = 3.804 (the median faulty distance). HI = 100% x clip((D_0 - D) / (D_0 - D_100), 0, 1).

Split	Healthy recordings (HI)	Faulty HI min / median / max	Faulty at or below 50%	ROC-AUC of the distance
train	100%, 100%, 100%, 100%, 100%, 100%, 100%, 100%	0 / 0 / 97%	45 / 56	1.000
val	100%, 100%, 100%, 94%	0 / 22 / 100%	20 / 28	0.955
test	0%, 100%, 0%, 100%	0 / 35 / 100%	15 / 28	0.527

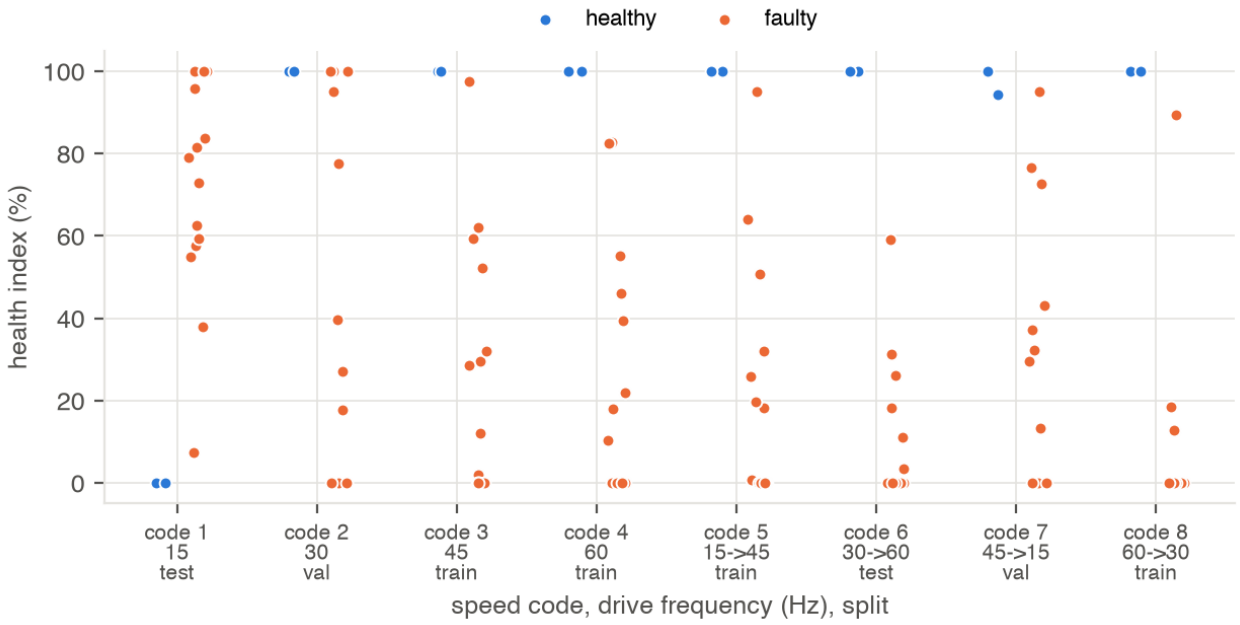


Figure 7. Health index of every recording by speed code. Many recordings sit exactly at 0% or 100% and overlap (the table above gives the counts). At 30 Hz and above the healthy motor sits at or near 100% and most faults far below; at 15 Hz (test) both healthy recordings score 0%.

Where it works and where it does not. Where the healthy reference is dense it works: at speed code 6 (test) both healthy recordings score 100% and the faulty median is 0%. Along the validation 45->15 Hz ramps (table below) it is correct from 22.5 Hz up, uninformative at 19.5 Hz and inverted at 16.5 Hz; at 15 Hz (test) both healthy recordings score 0%. Some faulty recordings also score 100% (3 in validation, 3 in test), so a 100% does not prove a healthy motor. Proposed, not yet decided: show no reading below a speed chosen on these validation windows (around 22 Hz), as a new experiment.

Window frequency (val, 45->15 Hz ramps)	Healthy windows' distance d	Faulty median d	Window ROC-AUC
16.5 Hz	5.10, 16.43	2.76	0.000
19.5 Hz	4.55, 2.03	2.65	0.500
22.5 Hz	3.27, 1.53	4.59	0.964
25.5 Hz	1.78, 0.81	5.52	1.000
28.5 Hz	0.76, 0.73	4.03	1.000
31.5 Hz	0.94, 1.01	3.37	1.000
34.5 Hz	0.75, 1.26	3.12	1.000
37.5 Hz	0.84, 0.81	3.07	1.000
40.5 Hz	0.87, 0.70	3.32	1.000
43.5 Hz	0.97, 0.50	3.46	1.000

Healthy distances above D_0 (3.80) give 0%; the healthy reference below 30 Hz rests on 2 training windows per frequency.

10. INT8: from the trained network to integers

The accelerator works in 8-bit integers. `export.py` converts E3 seed 42 with the hardware's own rounding (checked against `mac.py`'s bit-exact model, `golden_mlp`). s_h,1 and s_h,2 are the two hidden layers' activation scales; s_w is a weight column's scale; m0 (a 16-bit multiplier) and shift (a right shift) are the integers that apply a real rescale on the chip, m0 / 2^{shift}.

Step	Rule	Value
Inputs	q = clip(round(x_std / s_x), -127, 127), one scale for all 22 inputs; s_x = the 99.9th percentile of the train inputs / 127 (chosen on validation over the maximum)	s_x = 0.0475667
Weights	per output channel: s_w = max|W| / 127, W_q = round(W / s_w)	-127 .. 127
Hidden layers	bias = round(b / (s_in x s_w)); m0 / 2^shift = s_in x s_w / s_h; the accelerator rounds halves up	s_h,1 = 0.07163, s_h,2 = 0.1317
Last layer	the chip returns the raw sum C; the ARM adds bias_last; one integer step = s_h,2 x s_w,last of a real logit	bias_last = -120, step = 0.000626296
Threshold	T_int = ceil(cut / (s_h,2 x s_w,last)); faulty if the sum of the last 10 logits >= 10 T_int	T_int = 1768

Split	FP32 [[TN,FP],[FN,TP]]	INT8 [[TN,FP],[FN,TP]]	ROC-AUC FP32 / INT8	Decisions changed
train	[[8, 0], [0, 56]]	[[8, 0], [0, 56]]	1.000 / 1.000	0
val	[[4, 0], [0, 28]]	[[4, 0], [0, 28]]	1.000 / 1.000	0
test	[[4, 0], [1, 27]]	[[4, 0], [1, 27]]	1.000 / 1.000	0

INT8 changes no 10-second decision in any split: the test result stays 27/28 found, 0/4 false alarms, ROC-AUC 1.000. Window logits move by 0.44 on average over all windows (1.09 on test). The largest changes, up to 48, are in healthy windows whose kurtosis is far beyond the input clip at 6 standard deviations: FP32 extrapolates those to very negative scores, INT8 clips them, so **in INT8 these healthy recordings sit much closer to the threshold**:

Healthy recording	FP32 score	INT8 score	Highest INT8 window
H_H_1_0	-27.07	-6.28	1.06
H_H_1_1	-20.06	-9.91	-0.44
H_H_7_0	-10.63	-7.41	-4.59
H_H_7_1	-12.32	-7.85	4.86

Scores are mean logits in real units against the cut 1.11. Healthy windows at or above the cut in INT8: H_H_7_1 window 9: 4.86 (FP32 -34.37); one second alone would raise an alarm there, the 10-s decision stays healthy. Every sum fits the accelerator's 22-bit accumulator.

11. On the chip

Rows per run	Passes	Model cycles	Time at 214.286 MHz	Array use
1 (one 1-s window, once a second)	22	566	2.64 us	3.4%
10 (the last 10 windows at once)	22	818	3.82 us	23.5%

A pass streams one 8 x 8 block of a layer's weights through the 8 x 8 array: 3 x 4 + 4 x 2 + 2 x 1 = 22 for this model, whatever the number of rows; extra rows only lengthen each pass, which is why 10 rows cost 252 cycles more than 1. The cycle counts come from `mlp_equation`, the cycle formula in `mac.py`, which the RTL's counters match. Array use = multiply-adds / (64 PEs x model cycles). For comparison, E2's 15 inputs take 474 and 690 cycles. Weights: 1,232 bytes in the accelerator's weight slots.

RTL check: every logit bit for bit. `rtl_check.py` runs the exported hex files through the accelerator's RTL (`tb/tb_mlp.sv`, Icarus Verilog 13.0, the ZedBoard parameters) and compares every raw sum, the last layer's INT8 inputs and the cycle counters with Python. All three runs pass (27 Oct, from docs/log.md):

Run	Result
All 1,280 windows, 64 rows per model run (20 runs)	PASS, 2,330 cycles per run (= equation)
Each of the 32 test recordings as one 10-row run	PASS, 818 cycles (= equation)
One recording's 10 windows one at a time	PASS, 566 cycles (= equation)
Decisions from the RTL's logits	identical in all three runs; equal to the exported test vectors

Not done yet: the model has not run on the board. The on-chip MLP itself passed its board tests (30/30) at 214.286 MHz on 24 Oct; running this trained model there is the next step.

12. Independent checks

Every part was rebuilt by checkers that wrote their own code from the written definitions; a second agent then tried to refute every defect they reported. None of their findings changed a result.

- **Ramps.** Two other trackers find the same 7 recordings; fitted times agree within 0.07 s; every window frequency equals the exact integral of its command.
- **Training.** A trainer written from the spec reproduces every weight, threshold, loss, score and metric of E1, E2 and E3 bit for bit, all seeds, once four details the spec left unstated (now in `train.py`'s docstring) were supplied.
- **Health index.** An own implementation agrees to within 4e-15.
- **INT8 and RTL.** Own quantization equal to every hex file, header value and test vector; the C header compiles without a warning; the RTL check fails on deliberate errors (it is not vacuous); the accelerator's stock regression passes on Icarus 13.
- **This report.** Its numbers were recomputed twice by independent agents (the model comparison, then an audit of this PDF) with no mismatch; a reader's review led to the clarifications in this version.

Defects found and fixed along the way: the model choice had been called 'validation only' (corrected in section 7); the health index had been described as working 'from 16.5 Hz up'; a rounding edge case in the export (no exported value was affected); and several wording and number errors in the log.

13. Limits and next steps

- **One healthy motor.** On accelerometers 1 and 2 it is quieter than every faulty recording, and it turns slightly faster than most of them (all of them at 60 Hz). The model may partly recognize this motor rather than the fault, and accelerometer 1 is what E3 adds. A recording of a second healthy motor, or of the same motor before and after a fault, is the only way to tell.
- **Development test set.** The test split guided E3's creation and the choice of model, so 27/28 is optimistic; the three-seed mean is the fairer figure. A new hold-out set would give a clean number.
- **Few healthy test recordings.** 0 false alarms in 4 means the false-alarm rate is only known to be below about 60%. More healthy recordings would tighten this most.
- **Low speed.** All misses are at 15 Hz, below every training window; the health index fails there, and in INT8 the healthy 15-Hz recordings sit closer to the threshold.
- **Threshold.** The rule puts the cut at the edge of the faulty validation scores; a cut in the middle of the gap would catch more 15-Hz faults, but choosing it now would use the test result.
- **Next.** Alex to confirm E3; run the exported model on the ZedBoard and compare the board's INT8 rows and logits with `export/E3/vectors/`; merge Alex_edit's 24 Oct log into this branch.

Appendix A. Every seed, validation and test

Exp.	Seed	Split	Acc.	Bal.	Prec.	Recall	Spec.	F1	MCC	AUC	Confusion
E0	42	val	0.969	0.875	0.966	1.000	0.750	0.982	0.851	0.991	[[3, 1], [0, 28]]
E0	42	test	0.688	0.821	1.000	0.643	1.000	0.783	0.429	0.875	[[4, 0], [10, 18]]
E0	43	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E0	43	test	0.562	0.750	1.000	0.500	1.000	0.667	0.333	0.768	[[4, 0], [14, 14]]
E0	44	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E0	44	test	0.625	0.786	1.000	0.571	1.000	0.727	0.378	0.893	[[4, 0], [12, 16]]
E1	42	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E1	42	test	0.594	0.768	1.000	0.536	1.000	0.698	0.355	0.929	[[4, 0], [13, 15]]
E1	43	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E1	43	test	0.594	0.554	0.895	0.607	0.500	0.723	0.072	0.562	[[2, 2], [11, 17]]
E1	44	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E1	44	test	0.594	0.768	1.000	0.536	1.000	0.698	0.355	0.973	[[4, 0], [13, 15]]
E2	42	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E2	42	test	0.875	0.929	1.000	0.857	1.000	0.923	0.655	1.000	[[4, 0], [4, 24]]
E2	43	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E2	43	test	0.594	0.554	0.895	0.607	0.500	0.723	0.072	0.580	[[2, 2], [11, 17]]
E2	44	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E2	44	test	0.844	0.911	1.000	0.821	1.000	0.902	0.604	1.000	[[4, 0], [5, 23]]
E3	42	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E3	42	test	0.969	0.982	1.000	0.964	1.000	0.982	0.878	1.000	[[4, 0], [1, 27]]
E3	43	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E3	43	test	0.906	0.946	1.000	0.893	1.000	0.943	0.714	1.000	[[4, 0], [3, 25]]
E3	44	val	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	[[4, 0], [0, 28]]
E3	44	test	0.969	0.982	1.000	0.964	1.000	0.982	0.878	1.000	[[4, 0], [1, 27]]

Confusion matrices are [[TN, FP], [FN, TP]] with faulty as the positive class (validation: 4 healthy, 28 faulty; test: 4 healthy, 28 faulty).

Appendix B. Files and commands

Path (in model/)	What it is
features.py, speed.py	statistics of every recording and window; the drive frequency per window
train.py E0 ... E3	trains seeds 42, 43, 44; weights/ and results/ per experiment
health.py	the health index (output 2): weights/H1, results/H1
export.py	INT8 export of E3 seed 42: export/E3 (hex files, mg_model.h, test vectors)
rtl_check.py	runs export/E3 on the RTL with Icarus Verilog and checks every number
report/make_report.py	this report
README.md	setup and the commands to run everything yourself
export/E3/mg_model.h	everything the board's C program needs, with the steps
../docs/log.md, section 12	every decision and result, in order, with the reasons